



CS 225

Data Structures

Volodymyr Kindratenko

Slides courtesy of Wade Fagen-Ulmschneider

Introduction

Prof. Volodymyr Kindratenko, UIUC

Prof. Yushi Cheng, ZJUI

Prof. Gaoang Wang, ZJUI

TAs

Wendao Yao

Licheng Xu

Kerui Zhang

Yuxuan She

Qi Li

Jincheng An

Everything about CS 225

[CS225:Data Structures-1106-LEC\(Spring 2026\)](#)

Information on:

Staff

Communications

Labs

MPs

Quizzes

Final Exam

Grading

Academic Integrity

Class Schedule

Lectures

15:30-17:00 (W) RC-2-101

13:00-14:30 (F) RC-2-101

Labs

18:00-19:00 Thursdays LTN-A 414, 401, 301, 325

Quizzes

19:00-20:00 Thursdays LTN-A 414, 401, 301, 325

Every other week

Final Exam

TBD

Workload and grades

- **100 points:** Weekly lab assignments
 - There are 14 weeks -- points above 100 are extra credit!
- **130 points**
 - 10 points for MP1, 20 points for the rest of the MPs; up to 35 points in EC
- **520 points**
 - short answer quizzes (theory/math-focused)
 - programming quizzes (coding focused)
 - 40 points first, 80 points each for next six
- **250 points:** Final Exam
- **Attendance**
 - 15 sign-ups in total, 2 points each
 - up to 30 points in EC

Usual cutoffs

Points	Minimum Grade
[870, ∞)	A-
[750, 870)	B-
[600, 750)	C-
[500, 600)	D
[0, 500)	F

Course Activities - Lectures

Week #	Wednesday	Thursday	Friday
1	<i>January 14</i> Intro, Classes		<i>January 16</i> Memory
2	<i>January 21</i> Heap, References, Parameters	<i>Jan. 22</i> Quiz 1	<i>January 23</i> The Rule of Three

Course Activities – Labs

Assignment

Due

lab_intro: Ineluctable Introduction

Sunday, January 25, 11:59pm

lab_debug: Disastrous Debugging

Sunday, March 1, 11:59pm

lab_inheritance: Insidious Inheritance

Sunday, March 8, 11:59pm

lab_gdb2: Gruesome GDB

Sunday, March 15, 11:59pm

lab_quacks: Spiteful Stacks, Questionable Queues

Sunday, March 22, 11:59pm

lab_trees: Tempestuous Trees

Sunday, March 29, 11:59pm

lab_huffman: Hazardous Huffman Codes

Sunday, April 5, 11:59pm

lab_avl: Awful AVL Trees

Sunday, April 12, 11:59pm

lab_btree: Belligerent BTrees

Sunday, April 19, 11:59pm

lab_hash: Hellish Hash Tables

Sunday, April 26, 11:59pm

lab_heaps: Happy Heaps

Sunday, May 3, 11:59pm

lab_dict: Devious Dictionaries

Sunday, May 10, 11:59pm

lab_graphs: Gory Graphs

Sunday, May 17, 11:59pm

lab_ml: Lazy Machine Learning

Sunday, May 24, 11:59pm

Course Activities - MPs

Assignment	Leading TA	Due (Extra credit)	Due (Final)
MP 1: Image Manipulation	TBD	N/A	Monday, March 2, 11:59pm
MP 2: Images and Stickers	TBD	Monday, March 9, 11:59pm	Monday, March 16, 11:59pm
MP 3: Images and Lists	TBD	Monday, March 23, 11:59pm	Monday, March 30, 11:59pm
MP 4: Flood Fill	TBD	Monday, April 6, 11:59pm	Monday, April 13, 11:59pm
MP 5: PhotoMosaic	TBD	Monday, April 20, 11:59pm	Monday, April 27, 11:59pm
MP 6: TBD		N/A	Monday, May 11, 11:59pm
MP 7: Mazes	TBD	Monday, May 18, 11:59pm	Monday, May 25, 11:59pm

Lecture Handouts

- Posted as a pdf file before the lecture
- A condensed version of the lecture slides
- Students are expected to download them and make notes on them during the lectures

CS 225
#21: Introduction
February 13, 2023
V. Kindratenko

Welcome to CS 225: "Data Structures"
Course Website: <https://wikit.illinois.edu/wiki/display/CS225>
...do not mix it with UIUC CS 225 course website...

Description: Data abstractions: elementary data structures (lists, stacks, queues, and trees) and their implementation using an object-oriented programming language. Solutions to a variety of computational problems such as search on graphs and trees. Elementary analysis of algorithms.

Instructors:
Volodymyr Kindratenko <kindrv@intl.i>, Prof.
Gaogang Wang <gaogangwang@intl.i>, Prof.
Zuozhu Liu <zhuoliu@intl.i>, Prof.

Hanwen (Howie) Liu <hanwen4@illinois.edu>
Chuxuan Hu <chuxuan.19@intl.i>
Jiarun Hu <jiarun.19@intl.i>
Yu Liu <yuliu@intl.i>
Zhixin Chen <zhixin.20@intl.i>
Yue Yuan <yueyuan.18@intl.i>

Lab Sections: As part of registering for CS 225, you have also registered for a lab section. **Labs start this week** and are held by your TA – your lab TA is the first person who you should contact if you're having any troubles with the course!

Coursework and Grading

A total of 1,000 points are available in CS 225, along with many opportunities to earn extra credit. The points are broken down in the following way:

- **100 points:** Weekly lab assignments
 - There are 14 weeks – points above 100 are extra credit!
- **130 points:** MPs (10 for MP1, 20 for MP2-7)
 - Early submissions of MPs earn up to +7 points/MP after MP1
- **520 points** (13 x 40pts): Weekly exams
 - 7 short answer exams (theory/math-focused)
 - 6 programming exams (coding focused)
- **250 points:** Final Exam

Final Course Grades

Your final course grade is determined by the number of points you earned during the semester:

Points	Grade	Points	Grade	Points	Grade
[970, ∞)	A+	[595, 670]	A	[900, 950]	A-
[870, 960]	A-	[585, 595]	B	[850, 890]	B-
[770, 860]	C+	[725, 770]	C	[700, 750]	C-
[670, 760]	D+	[635, 670]	D	[600, 630]	D-
		[600, 0]	F		

We never curve individual exam or assignment scores; instead, if necessary, we may lower the points required for each grade cutoff to be lower than the stated cutoff. However, this has not been necessary in recent semesters. In no case will we raise the cutoff.

Contacting Us:

We want to do everything we can to make sure you have a great time learning data structures. There's a lot of ways to reach out if you get stuck:

1. **Course Piazza:** Online Q&A discussion forum to ask and answers from your peers and course staff. Join it!
2. **Open Office Hours:** TAs will be holding office hours every week, starting from this week. Great way to get help on MPs, labs, and technical content.
Location: Check CS 225's wld for place
Times: Check CS 225's wld for time
3. **Lab TA:** Your Lab TA is your primary administrative contact for the course. You will see him each week in lab and can reach him via e-mail. However, no coding assistance will be provided via e-mail (use office hours).

Variables and Classes in C++

Every variable is defined by four properties:

- 1.
- 2.
- 3.
- 4.

And every variable is one of two types:

int myFavoriteInt;	Sphere myFavoriteSphere;
char grade = 'A';	Cube cube;
double gamma = 0.653;	Grade courseGrade;

Creating New Types

In data structures, we will be learning and creating new types of structures to store data. We will start simply – by the end, we will have types we built being the building blocks for new types!

Big Idea: Encapsulation

Encapsulation principles:

sphere.h	sphere.cpp
1	1
2	2
3	3
4	4
5	5

Our first class:

sphere.h	sphere.cpp
1 #ifndef SPHERE_H	1 #include "sphere.h"
2 #define SPHERE_H	2
3	3 double
4 class Sphere {	4 Sphere::getRadius() {
5 public:	5
6	6 }
7	7
8	8
9	9
10	10
11 private:	11
12	12
13	13
14 }	14
15	15
16	16
17 #endif	

Key Concepts in C++ Classes:

Every class will be made up of common key pieces:

1. `sphere.h:1 :2 and :17`
2. `sphere.h:4..15`
3. `sphere.cpp:1`
4. `sphere.cpp:3`

CS 225 – Things To Be Doing:

1. Visit our course wld
2. Join the CS 225 Piazza
3. Lab section start this week – check when/where your lab is at
4. First MP is released soon, due Monday after next

Programming Environment

- Labs, MPs, **quizzes**, and final exam all require a Linux environment
 - See first lab for details how to setup it on your own computer.
- We expect all students to setup a working Linux environment on their own laptops:
 - Linux, particularly Ubuntu, but any other modern Linux environment should work
 - On Windows, the preferred way is to install WSL
 - On Mac, need Xcode and some other tools, particularly SVN
 - You can also continue using ECE 120 VM, it has all the necessary tools.



What is this course about?





ECE 220

CS 126

CS 125

CS 225

CS 173

MATH 213

Variables in C++

```
int myFavoriteInt;
```

```
char grade = 'A';
```

```
double gamma = 0.653;
```

```
Cat fiona, mia;
```

```
Cube rubix;
```

```
Person wade;
```

Encapsulation

Encapsulation is a separation of interface from implementation.

Two important side effects:

- 1st: it hides complexity
- 2nd: allows private data to be private

Object-oriented languages allow to expose the interface, but hide the logic. C++ embraces encapsulation at the language level.

We will define the interface in the .h file and will provide the implementation in .cpp file.

Encapsulation

`sphere.h`

`sphere.cpp`

sphere.h

```
1 #ifndef SPHERE_H
2 #define SPHERE_H
3
4 class Sphere {
5     public:
6
7
8
9
10
11
12     private:
13
14
15 };
16
17 #endif
18
19
20
```

sphere.cpp

```
1 #include "sphere.h"
2
3 double Sphere::getRadius() {
4
5
6 }
7
8
9
10
11
12
13
14
15
16
17
18
19
20
```

sphere.h

```
4 class Sphere {
5     public:
6         double getRadius();
```

sphere.h

```
1 #ifndef SPHERE_H
2 #define SPHERE_H
3
4 class Sphere {
5     public:
6         double getRadius();
7
8
9
10
11
12     private:
13         double r_;
14
15 };
16
17 #endif
18
19
20
```

sphere.cpp

```
1 #include "sphere.h"
2
3 double Sphere::getRadius() {
4     return r_;
5
6 }
7
8
9
10
11
12
13
14
15
16
17
18
19
20
```

Namespaces

There are probably many other Sphere classes out there, so, how do we differentiate them from **our** Sphere?

We can use the concept of **namespaces** to isolate one collection of classes from another:

- Namespaces provide an organization of multiple classes
- We will put our Sphere class into sc225 namespace
 - namespace cs225 { Sphere class goes here }

Namespaces

cs225

Sphere

PNG

HSLAPixel

std

cout

vector

queue

...

...

sphere.h

```
1 #ifndef SPHERE_H
2 #define SPHERE_H
3
4 namespace cs225 {
5     class Sphere {
6     public:
7         double getRadius();
8         double getVolume();
9
10
11
12
13     private:
14         double r_;
15
16     };
17 }
18
19 #endif
20
```

sphere.cpp

```
1 #include "sphere.h"
2
3 namespace cs225 {
4     double Sphere::getRadius() {
5         return r_;
6     }
7
8     double Sphere::getVolume() {
9         return (4 * r_ * r_ * r_ *
10             3.14159265) / 3.0;
11     }
12 }
13
14
15
16
17
18
19
20
```

main.cpp

```
1 #include "sphere.h"
2 #include <iostream>
3
4 int main() {
5     cs225::Sphere s;
6     std::cout << "Radius: " << s.getRadius() << std::endl;
7     return 0;
8 }
```

Compile like this:

```
g++ main.cpp sphere.cpp -o sphere
```

main.cpp

```
1 #include "sphere.h"
2 #include <iostream>
3
4 using namespace std;
5 using namespace cs225;
6
7 int main() {
8     Sphere s;
9     cout << "Radius: " << s.getRadius() << endl;
10    return 0;
11 }
```


Constructor

Constructor is the *first function that is called* when a new object of a given class is created

- This is where we will initialize the values, or state, of the class object

C++ class requires the constructor

- But in our sphere class we have not provided one!?

If you do not define it, C++ provides an *automatic default constructor*

- This is a default constructor, which means it has no parameters
- All it does is it sets all variables to the default values
- This only happens when you do not provide any other constructors!

sphere.h

```

1  #ifndef SPHERE_H
2  #define SPHERE_H
3  namespace cs225 {
4      class Sphere {
5      public:
6          Sphere();
7          double getRadius();
8          double getVolume();
9
10
11
12
13     private:
14         double r_;
15
16     };
17 }
18
19 #endif
20

```

sphere.cpp

```

1  #include "sphere.h"
2  namespace cs225 {
3      Sphere::Sphere() {
4          r_ = 1;
5      }
6
7
8      double Sphere::getRadius() {
9          return r_;
10     }
11
12     double Sphere::getVolume() {
13         return (4 * r_ * r_ * r_ *
14             3.14159265) / 3.0;
15     }
16 }
17
18
19
20

```

sphere.h

```

1  #ifndef SPHERE_H
2  #define SPHERE_H
3  namespace cs225 {
4      class Sphere {
5      public:
6          Sphere(double r);
7          double getRadius();
8          double getVolume();
9
10
11
12
13     private:
14         double r_;
15
16     };
17 }
18
19 #endif
20

```

sphere.cpp

```

1  #include "sphere.h"
2  namespace cs225 {
3      Sphere::Sphere(double r) {
4          r_ = r;
5      }
6
7
8      double Sphere::getRadius() {
9          return r_;
10     }
11
12     double Sphere::getVolume() {
13         return (4 * r_ * r_ * r_ *
14             3.14159265) / 3.0;
15     }
16 }
17
18
19
20

```

sphere.h

sphere.cpp

```

1 #ifndef SPHERE_H
2 #define SPHERE_H
3 namespace cs225 {
4     class Sphere {
5     public:
6         Sphere(double r);
7         double getRadius();
8         double getVolume();
9
10
11
12
13
14
15
16
17
18
19
20

```

```

1 #include "sphere.h"
2 namespace cs225 {
3     Sphere::Sphere(double r) {
4         r_ = r;
5     }
6
7
8     double Sphere::getRadius() {
9
10
11
12
13
14
15
16
17
18
19
20

```

main.cpp

```

1 #include "sphere.h"
2 #include <iostream>
3
4 using namespace std;
5 using namespace cs225;
6
7 int main() {
8     Sphere s;    /* this will fail to compile! Why? */
9     cout << "Radius: " << s.getRadius() << endl;
10    return 0;
11 }

```

sphere.h

sphere.cpp

main.cpp

```

1 #ifndef SPHERE_H
2 #define SPHERE_H
3 namespace cs225 {
4     class Sphere {
5     public:
6         Sphere(double r);
7         double getRadius();
8         double getVolume();

```

```

1 #include "sphere.h"
2 namespace cs225 {
3     Sphere::Sphere(double r) {
4         r_ = r;
5     }
6 }

```

```

9     #include "sphere.h"
10    #include <iostream>
11
12    using namespace std;
13    using namespace cs225;
14
15    int main() {
16        Sphere s(22);
17        cout << "Radius: " << s.getRadius() << endl;
18        return 0;
19    }

```

```

7     double Sphere::getRadius() {
8         return r_;
9     }
10
11     double Sphere::getVolume() {
12         return (4.0/3.0)*r_*r_*r_;
13     }

```

sphere.h

sphere.cpp

main.cpp

```

1 #ifndef SPHERE_H
2 #define SPHERE_H
3 namespace cs225 {
4     class Sphere {
5     public:
6         Sphere();
7         Sphere(double r);
8         double getRadius();

```

```

1 #include "sphere.h"
2 namespace cs225 {
3     Sphere::Sphere() {
4         r_ = 1;
5     }
6     Sphere::Sphere(double r) {
7         r_ = r;
8     }

```

```

1 #include "sphere.h"
2 #include <iostream>
3
4 using namespace std;
5 using namespace cs225;
6
7 int main() {
8     Sphere s; /* this will now compile */
9     cout << "Radius: " << s.getRadius() << endl;
10    return 0;
11 }

```

```

20 #endif

```

```

20

```

Pointers

powerful & dangerous

```
Sphere s1;
```

```
s1.getRadius();
```

```
Sphere* s2 = &s1;
```

-same as-

```
Sphere *s2 = &s1;
```

```
s2->getRadius();
```


CS 225 – Things To Be Doing

Labs Sections (Discussions) Start this Week...

Make sure to attend your lab section (discussion) this week!

MP1 is coming soon...

Due: Monday, March 2nd (after the winter break)

Office Hours are Starting Up...

See BB, there are many office hours offered

Also ask questions any time on Piazza

See you Friday!